



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 04

Objective & Lecture Mapping: In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment [cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

Part 1: Practical Implementation — Building the A* Agent

Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, $h(n)$, which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called `Week_04` in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class [cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula $h(n) = |x_1 - x_2| + |y_1 - y_2|$ and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula $h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

Step 1.2: Implementing A* Search

A* evaluates nodes by combining the path cost so far, $g(n)$, and the estimated cost to the goal, $h(n)$. The evaluation function is $f(n) = g(n) + h(n)$.

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses $g(n)$, your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, $g(n) = 0$. Calculate $h(n)$ using your chosen heuristic method, and set $f(n) = g(n) + h(n)$.
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos` to `reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate $g_{\text{new}} = g_{\text{current}} + 1$, h_{new} using your heuristic, and $f_{\text{new}} = g_{\text{new}} + h_{\text{new}}$. Push the new tuple to the priority queue.

Step 1.3: Integrating A* into the Agent's Decision Loop

Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 05, complete the following analytical questions.

1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?

UCS only uses the actual path cost so far, $g(n)$, to decide which node to explore next, it always picks the node with the lowest cost from the start, without any idea of how far the goal might be. A* improves on this by using $f(n) = g(n) + h(n)$, which combines the actual path cost $g(n)$ with a heuristic estimate $h(n)$ of how far the node is from the goal. This means A* is "smarter" — it prioritizes nodes that are not only cheap to reach but also appear to be closer to the goal.

in our code:

UCS pushes (cost, position, path) into the priority queues only $g(n)$. A* pushes (f_cost, g_cost, position, path) where $f_cost = g_cost + \text{heuristic}(\text{position}, \text{goal})$. This is why A* explores fewer nodes than UCS and finds the optimal path faster.

2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?

A heuristic is admissible if it never overestimates the actual cost to reach the goal, it's always optimistic or exactly right. Manhattan Distance is admissible for our grid because the agent can only move in 4 directions (Up, Down, Left, Right), with each step costing 1. The shortest possible path between any two points on this grid (ignoring walls) is exactly $|x_1 - x_2| + |y_1 - y_2|$, which is exactly what Manhattan Distance calculates. Since walls can only make the real path

longer (never shorter), Manhattan Distance will always be less than or equal to the true cost. It never overestimates, so it is admissible. If the heuristic was not admissible (i.e., it overestimates), A* could miss the optimal path. It might think a node is too far from the goal and skip it, even though that node was actually on the shortest path. The algorithm would still find a path, but it would no longer guarantee that the path is the shortest/cheapest one.

3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?

No, Manhattan Distance would not be admissible with 8-way diagonal movement. Consider a simple example: moving from (0, 0) to (3, 3).

Manhattan Distance says the cost is $|0-3| + |0-3| = 6$ steps.

But with diagonal movement, the agent can move diagonally in just 3 steps (going one step diagonally covers both x and y at once).

Since Manhattan returns 6 but the actual cost is only 3, it overestimates the true cost. This violates admissibility, and A* would no longer guarantee finding the optimal path. We should switch to Euclidean Distance, which calculates the straight-line distance: $\sqrt{(x_1-x_2)^2 + (y_1-y_2)^2}$. For the same example, Euclidean gives $\sqrt{9+9} \approx 4.24$, which is $\leq$ the actual diagonal cost, so it never overestimates and remains admissible. This is actually why we implemented both heuristics in our `astar_search` method with the `heuristic_type` parameter, so we can easily switch between 'manhattan' (for 4-way grids) and 'euclidean' (for 8-way grids).

4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.

A stronger heuristic would be to use the Minimum Spanning Tree (MST) cost of the remaining food positions plus the distance from the agent to the nearest food node in that tree.

Why the closest-food heuristic is weak: It only considers the distance to one food item and completely ignores the rest. After eating the closest food, the agent re-plans from scratch, which may lead to inefficient back-and-forth movement across the grid.

Proposed stronger heuristic:

Take the agent's current position and all remaining food positions.

Compute the MST connecting all the remaining food items using Manhattan distances as edge weights.

Add the Manhattan distance from the agent to the closest food item in the MST. The heuristic value $h(n) = (\text{distance to nearest food}) + (\text{total MST cost of remaining food})$.

Why this works: The MST represents the minimum total distance needed to connect all remaining food items together. Since the agent must visit every food item at least once, the MST cost is a lower bound on the actual travel cost — meaning it never overestimates, so it remains admissible. At the same time, it gives a much tighter estimate than just looking at the single closest food, which makes A* expand far fewer nodes and find a more efficient overall path.

Example: If there are 5 food items scattered across the grid, the closest-food heuristic might return $h(n) = 3$ (distance to nearest food), while the MST heuristic could return $h(n) = 3 + 18 = 21$, which is a much more informed estimate of the remaining work and guides A* more effectively.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation



FACULTY OF COMPUTING